

DSA STUDY BLUEPRINT SERIES

37. Recursion Part3 Backtracking Subsets

Generating Subsets and Power Sets using Recursion

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Power Set:** A set of all possible subsets (size 2^N).
- **Include-Exclude:** Decisions at each step: include current element or exclude it.

Memory & Execution Blueprint

Decision Tree splits:

[] $\rightarrow$ Include 1 $\Rightarrow$ [1] • Exclude 1 $\Rightarrow$ []

C++ Implementation Reference

Standard code layout and syntax

```
void getSubsets(vector& nums, int i, vector& curr, vector<vector>& ans) {  
    if (i == nums.size()) { ans.push_back(curr); return; }  
    curr.push_back(nums[i]); getSubsets(nums, i + 1, curr, ans); // Include  
    curr.pop_back(); getSubsets(nums, i + 1, curr, ans); // Exclude  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(2^N)$
Space Complexity	$O(N)$ (Recursion Stack)

Key Practices & Gotchas

- **Important:** Ensure to backtrack (pop_back) to revert the state after a recursive call.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive decision tree transitions generating subsets for [1, 2] :

Decision Level	Element choice	Recursive action	Subsets array state
i = 0	Include 1	Recurse level 1	[[1]]
i = 1	Exclude 2	Base case hit	Add subset [1] to results

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **State Reversals:** Essential backtrack actions pop choice selections to restore node states before adjacent searches.
- **Pruning:** Skip duplicate recursive paths by checking elements duplicates before branching.
- **Related LeetCode Problems:** #78 Subsets, #90 Subsets II, #46 Permutations.